

Nonlinear Model Predictive Control with *acados*

November, 2025

1 Introduction

In this notebook, we will see an example of how to implement a Nonlinear Model Predictive Control (NMPC) controller with the open source software *acados*.

Detailed information about *acados* can be found in the [official documentation page](#) and in the related published paper:

R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen and M. Diehl, “acados — a modular open-source framework for fast embedded optimal control,” *Mathematical Programming Computation*, 2021.

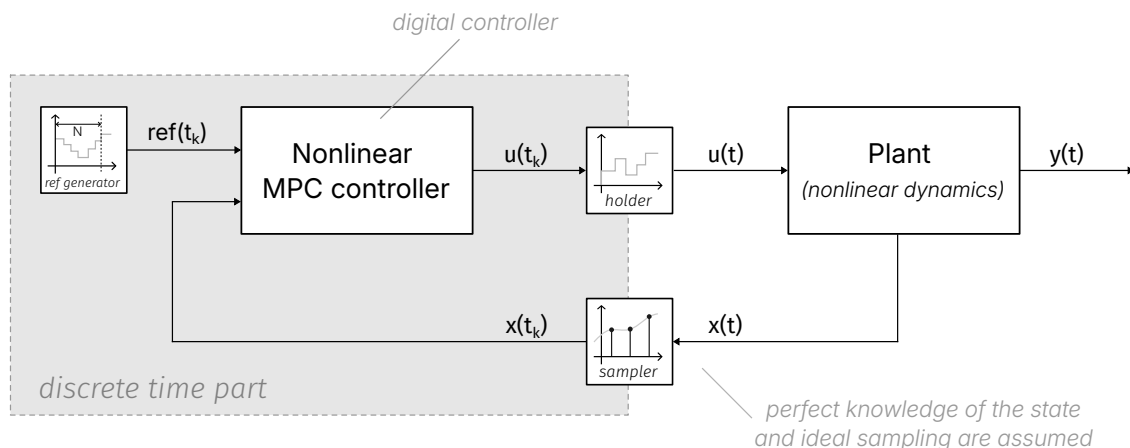
which is also available online at

- <https://arxiv.org/abs/1910.13753> (pre-print)
- <https://publications.syscop.de/Verschueren2021.pdf>

In particular, we will use the Python interface, which is given by the package `acados_template`.

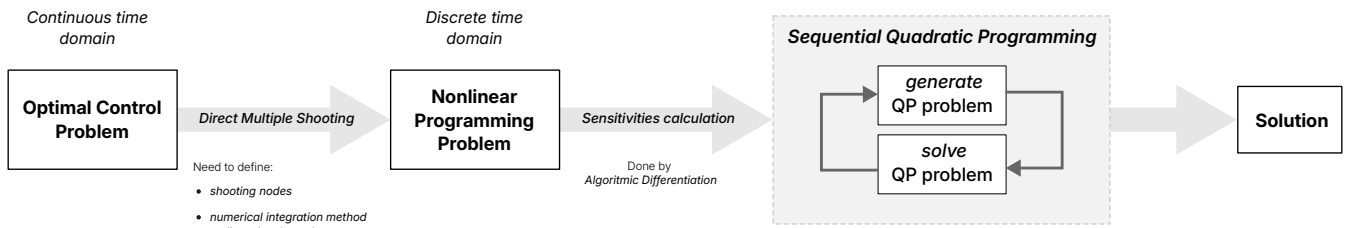
2 Setup definition

The **control loop** that we will consider is the following:

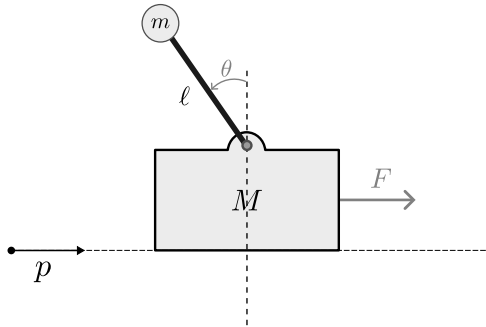


and we will make use of the tools provided by *acados* to implement both the Nonlinear MPC controller and to simulate the nonlinear system dynamics.

Concerning the **Nonlinear MPC controller**, the algorithmic pipeline that we will consider is the following:



Instead, about the **system dynamics**, we will consider the *Inverted Pendulum* system:



$$\begin{cases} \dot{p} = v \\ \dot{v} = \omega \\ \dot{\theta} = \omega \\ \dot{\omega} = \frac{-ml \sin(\theta)\omega^2 + mg \cos(\theta) \sin(\theta) + F}{M + m - m \cos^2 \theta} \\ \dot{\omega} = \frac{-ml \cos(\theta) \sin(\theta)\omega^2 + F \cos(\theta) + (M + m)g \sin(\theta)}{l(M + m - m \cos^2(\theta))} \end{cases}$$

and our **control task** will be to perform a *swing-up maneuver*, that is, start with the pendulum pointing downwards, bring it upwards and keep it stable in that position.

3 Getting started

The first operation that we need to perform is to import all the necessary packages:

```
[ ]: from acados_template import AcadosModel, AcadosOcp, AcadosOcpSolver, AcadosSim, \
    ↪AcadosSimSolver
import casadi as ca
import numpy as np
from scipy.linalg import block_diag
from utils import piecewise_constant, plot_results, plot_pred_traj, plot_cpt, plot_grid, \
    ↪inverted_pendulum_animation
from tqdm import tqdm

# to make the figures interactive
import matplotlib
%matplotlib widget

# to visualize videos in the notebook
from IPython.display import Video
```

3.1 Encoding the system model

Then, we need to encode the continuous time model of our system into an `AcadosModel` object. This will be used both for the design of the Nonlinear MPC controller and to simulate the system.

Notice that we will use [CasADi](#) symbolic variables (`casadi.SX.sym`) in the definition of the model: this will make possible automatic differentiation to compute sensitivities.

```
[ ]: def get_inverted_pendulum_model(M = 1, m = 0.1, l = 0.8, g = 9.81)-> AcadosModel:

    #set model name
    model_name = "inverted_pendulum"

    #the physical parameters of the system are passed as parameters
    # g - gravity acceleration [m/s**2]
    # M - cart mass [kg]
    # m - pendulum mass [kg]
    # l - pendulum length [m]

    # setup states and controls symbolic variables
    p      = ca.SX.sym('p', 1, 1)
    theta  = ca.SX.sym('theta', 1, 1)
    v      = ca.SX.sym('v', 1, 1)
    omega  = ca.SX.sym('omega', 1, 1)

    x = ca.vertcat(p, theta, v, omega)
    u = ca.SX.sym('u', 1, 1)

    # setup symbolic variables for xdot (to be used with IRK integrator)
    p_dot   = ca.SX.sym('p_dot', 1, 1)
    theta_dot = ca.SX.sym('theta_dot', 1, 1)
    v_dot    = ca.SX.sym('v_dot', 1, 1)
    omega_dot = ca.SX.sym('omega_dot', 1, 1)

    xdot = ca.vertcat(p_dot, theta_dot, v_dot, omega_dot)

    # define dynamics
    a = -m*l*ca.sin(theta)*omega**2+m*g*ca.cos(theta)*ca.sin(theta)+u
    b = -m*l*ca.cos(theta)*ca.sin(theta)*omega**2+u*ca.cos(theta)+(M+m)*g*ca.sin(theta)
    c = M+m-m*(ca.cos(theta))**2

    # explicit ODE right hand side (to be used with ERK integrator)
    f_expl = ca.vertcat(v,
                        omega,
                        a/c,
                        b/(l*c))

    # implicit dynamics (to be used with IRK integrator)
    f_impl = xdot - f_expl

    # create acados model and fill in all the required fields
    model = AcadosModel()

    model.f_impl_expr = f_impl
    model.f_expl_expr = f_expl
    model.x = x
    model.xdot = xdot
```

```

model.u = u
model.name = model_name

return model

```

3.2 Defining the Optimal Control Problem

The next step is to define the continuous time Optimal Control Problem (OCP) and to encode it into an `AcadosOcp` object.

However, despite its name, an `AcadosOcp` object actually contains not only the OCP but also all the information necessary to discretize it with *direct multiple shooting* into a Nonlinear Programming (NLP) problem and then to solve this latter with Sequential Quadratic Programming.

To better understand what we need, let us recall that, considering a fairly general formulation (which is enough for our purposes), we can write the OCP and the NLP problem as:

$$\begin{array}{ccc}
 \min_{x(\cdot), u(\cdot)} \int_0^T \ell(x(t), u(t)) dt + M(x(T)) & & \min_{\substack{x_0, \dots, x_N \\ u_0, \dots, u_{N-1}}} \sum_{k=0}^{N-1} \ell(x_k, u_k) \cdot (t_{k+1} - t_k) + M(x_N) \\
 \text{subject to } x(0) = \hat{x}_0 & \xrightarrow{\text{Direct Multiple Shooting}} & \text{subject to } x_0 = \hat{x}_0 \\
 \dot{x}(t) = f(x(t), u(t)), \quad \forall t \in [0, T] & & x_{k+1} = \phi(x_k, u_k), \quad k = 0, \dots, N-1 \\
 g(x(t), u(t)) \leq 0, \quad \forall t \in [0, T] & & g(x_k, u_k) \leq 0, \quad k = 0, \dots, N-1 \\
 g_f(x(T)) \leq 0 & & g_f(x_N) \leq 0
 \end{array}$$

So, in addition to all the elements characterizing the original OCP, to construct the NLP we will need to specify:

- how to take the **shooting time instants**
 - ⇒ we divide T in N intervals that in the simplest case will be of equal amplitude T/N (and we will also set this value for the sample time of the controller)
- the **numerical integration method** to use for discretizing the dynamics
 - ⇒ we will consider *4-th order Runge-Kutta method*, both explicit and implicit

Then, once that we have constructed the NLP, given that the sensitivity computation is performed automatically, in order to solve it with Sequential Quadratic Programming, we need to define:

- the type of **approximation for the Hessian** of the Lagrangian
 - ⇒ we will consider the *Gauss-Newton* approximation, which involves only first-order differentiations
- an **initialization** for the problem, a first guess to start the iterative algorithm
- the **QP solver**, i.e. the software used to solve each QP problem
 - ⇒ `acados` supports different external QP solvers, and in particular we will consider [HPIPM](#) and [qpOASES](#)
- the **value of parameters** (e.g., maximum number of iterations, tolerances, etc.) for both for the QP solver and the SQP algorithm
 - ⇒ these are set to a default (reasonable) value, but we need to be aware of these choices

Finally, an aspect which needs to be discussed is how to encode the elements of the starting OCP.

- Particularly, for the **cost function definition**, `acados` consents to specify separately the type of the stage cost and the terminal cost (which is optional) and here we will consider two possibilities (*simplified for our case* – for the general definition refer to the documentation):

- the type `LINEAR_LS` defines

$$\ell(x, u) = \frac{1}{2} \|V_x x + V_u u - y_{ref}\|_W^2, \quad M(x) = \frac{1}{2} \|V_x^e x - y_{ref}^e\|_{W^e}^2,$$

where

- * $y_{ref} \in \mathbb{R}^{n_y}$ and $y_{ref}^e \in \mathbb{R}^{n_y^e}$ represent the reference
- * the matrices $V_x \in \mathbb{R}^{n_y \times n_x}$, $V_u \in \mathbb{R}^{n_y \times n_u}$ map the state x and the control u to the reference space
- * the matrix $V_x^e \in \mathbb{R}^{n_y^e \times n_x}$ maps the terminal state to the terminal reference space
- * the matrices $W \in \mathbb{R}^{n_y \times n_y}$ and $W^e \in \mathbb{R}^{n_y^e \times n_y^e}$ provide the weights used to compute the norm
- the type `NONLINEAR_LS` instead defines

$$\ell(x, u) = \frac{1}{2} \|y(x, u) - y_{ref}\|_W^2, \quad M(x) = \frac{1}{2} \|y^e(x) - y_{ref}^e\|_{W^e}^2,$$

and hence it allows to specify

- * a generic nonlinear mapping $y(x, u) : \mathbb{R}^{n_x \times n_u} \rightarrow \mathbb{R}^{n_y}$ for the state and control to the reference space
- * a generic nonlinear mapping $y^e(x) : \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_y^e}$ for the state to the terminal reference space.

Once that stage and terminal cost are set, `acados` by default discretizes the integral in the original cost function using *Euler* method (so, as shown in the NLP above). Advanced options are available for the `NONLINEAR_LS` case.

Hence, if we want to encode the following cost:

$$\int_0^T [\tilde{x}^T(t) Q \tilde{x}(t) + \tilde{u}^T(t) R \tilde{u}(t)] dt + \tilde{x}^T(T) P_f \tilde{x}(T)$$

$$\tilde{x}(t) = x(t) - x_{ref}(t), \quad \tilde{u}(t) = u(t) - u_{ref}(t), \quad Q \in \mathbb{R}^{n_x \times n_x}, \quad R \in \mathbb{R}^{n_u \times n_u}, \quad P_f \in \mathbb{R}^{n_x \times n_x},$$

we can do it by choosing

$$W = \text{blkdiag}\{Q, R\}, \quad W^e = P_f$$

$$y_{ref}(x(t), u(t)) = \begin{bmatrix} x_{ref}(t) \\ u_{ref}(t) \end{bmatrix} \in \mathbb{R}^{n_x + n_u}, \quad y_{ref}^e(x(t)) = x_{ref}(t) \in \mathbb{R}^{n_x}$$

and

- for `LINEAR_LS` type

$$V_x = \begin{bmatrix} I_{n_x} \\ 0_{n_u \times n_x} \end{bmatrix}, \quad V_u = \begin{bmatrix} 0_{n_x \times n_u} \\ I_{n_u} \end{bmatrix}, \quad V_x^e = I_{n_x}$$

– for NONLINEAR_LS type

$$y(x, u) = \begin{bmatrix} x \\ u \end{bmatrix} \in \mathbb{R}^{n_x + n_u}, \quad y^e(x) = x \in \mathbb{R}^{n_x}$$

- Defining the **system dynamics** is straightforward, as we only need to set the parameter `model` of the `AcadosOcp` object to the `AcadosModel` which we saw before how to create.

Note: `acados` actually gives also the possibility to specify directly a *discrete* dynamics within an `AcadosModel` object.

- Concerning the **path and terminal constraints**, for now we will consider only *bounds*, i.e. constraints of type

$$lb_{x_i} \leq x_i \leq ub_{x_i}$$

$$lb_{u_i} \leq u_i \leq ub_{u_i}$$

on the generic i -th component of the state and control. Defining them is straightforward and we will see how to do it directly in the code.

In particular, for our example we will consider

$$-20 \text{ N} \leq u \leq 20 \text{ N}$$

which accounts for an actuator limitation.

```
[ ]: def create_ocp_solver_description(model, N, T, x0) -> AcadosOcp:

    # create ocp object to formulate the OCP
    ocp = AcadosOcp()

    # define system dynamics model
    ocp.model = model

    # set prediction horizon:
    # tf - prediction horizon length [s]
    # N - number of intervals in which the prediction horizon is divided
    ocp.solver_options.tf = T
    ocp.solver_options.N_horizon = N

    # NOTE: on older acados versions, use instead
    # ocp.dims.N = N

    # get state, control and cost dimensions
    nx = model.x.rows()
    nu = model.u.rows()

    ny = nx + nu
    ny_e = nx

    # define cost type
    ocp.cost.cost_type = 'LINEAR_LS'
    ocp.cost.cost_type_e = 'LINEAR_LS'

    # define cost weight matrices
```

```

Q = np.diag([10, 10, 0.1, 0.1])
R = 0.01

ocp.cost.W = block_diag(Q, R)
ocp.cost.W_e = T/N*Q

# define matrices characterizing the cost
ocp.cost.Vx = np.vstack((np.eye(nx), np.zeros((nu, nx))))
ocp.cost.Vu = np.vstack((np.zeros((nx, nu)), np.eye(nu)))
ocp.cost.Vx_e = np.eye(nx)

# alternatively, for the NONLINEAR_LS cost type
#ocp.model.cost_y_expr = ca.vertcat(model.x, model.u)
#ocp.model.cost_y_expr_e = model.x

# initialize variables for reference
ocp.cost.yref = np.zeros((ny,))
ocp.cost.yref_e = np.zeros((ny_e,))

# set constraints
ocp.constraints.lbu = np.array([-20])
ocp.constraints.ubu = np.array([20])
ocp.constraints.idxbu = np.array([0])

# initialize constraint on initial condition
ocp.constraints.x0 = x0

# set solver options
ocp.solver_options.qp_solver = "PARTIAL_CONDENSING_HPIPM" #FULL_CONDENSING_QPOASES, □
↪PARTIAL_CONDENSING_HPIPM
ocp.solver_options.hessian_approx = "GAUSS_NEWTON"
ocp.solver_options.integrator_type = "ERK" #ERK, IRK
ocp.solver_options.nlp_solver_type = "SQP" #SQP, SQP_RTI

# to configure partial condensing
#ocp.solver_options.qp_solver_cond_N = int(N/10)

# some more advanced settings (refer to the documentation to see them all)
# - maximum number of SQP iterations (default: 100)
ocp.solver_options.nlp_solver_max_iter = 100
# - maximum number of iterations for the QP solver (default: 50)
ocp.solver_options.qp_solver_iter_max = 50

# - configure warm start of the QP solver (0: no, 1: warm start, 2: hot start)
# (depends on the specific solver)
ocp.solver_options.qp_solver_warm_start = 0

return ocp

```

3.3 Performing simulation

3.3.1 Preliminaries

Since we are going to run different simulations, we define the following function to clean the `AcadosSimSolver` and `AcadosOcpSolver` objects generated in previous executions. Indeed, while in Python such variables would be overwritten, this would not affect the generated and compiled C code which is then actually run.

```
[ ]: def clear_previous_simulation():
    variables = globals().copy()
    for v in variables:
        if (type(variables[v]) is AcadosSimSolver or type(variables[v]) is
↪AcadosOcpSolver):
            del globals()[v]
```

Additionally, we will use the following function to check the consistency of the time parameters (i.e., the simulation fundamental step size, the controller sample time and the simulation final time instant)

```
[ ]: def compute_num_steps(ts_sim, Ts, Tf):

    # check consistency
    if not (Ts/ts_sim).is_integer():
        raise ValueError("The sample time Ts has to be an integer multiple of the
↪simulation time step ts_sim")
    if not (Tf/ts_sim).is_integer():
        raise ValueError("The simulation time Tf has to be an integer multiple of the
↪simulation time step ts_sim")

    # compute the number of simulation steps
    N_steps = int(Tf/ts_sim)

    # compute the number of steps for the discrete-time part of the loop
    N_steps_dt = int(Tf/Ts)

    # number of simulation steps every which to update the discrete-time part of the loop
    n_update = int(Ts/ts_sim)

    return (N_steps, N_steps_dt, n_update)
```

3.3.2 Setup simulation parameters

Now we need to define the parameters that are necessary to run the simulation. In particular, we need to fix:

- t_{sim} – simulation fundamental time step
- x_0 – initial condition
- N – number of shooting time intervals
- T_s – controller sample time

⇒ we assume the prediction horizon length $T = N \cdot T_s$

- reference signal corresponding to our task
- model of the system used for simulation

Furthermore, there are two decisions that we need to take:

- **how to provide the reference signal:** at every discrete time instant k , we will need to provide the controller with the reference signal *over all the prediction horizon*
 - if we consider the reference to be available in advance (so-called *reference look-ahead* or *reference preview*), at the time instant k we can define the reference using the samples from k to $k + N$ of the reference signal
 - ⇒ this implies that, also at the final time instant, we need to have N additional samples
 - ⇒ we can extend the signal with N replicas of the last sample
 - if at time instant k we assume to know only the current sample of the reference signal, we can construct a *constant* reference with replicas of it
- **how to initialize the problem:** at every discrete time instant k , we also need to provide an initial guess for the problem, that is for the value of the state at the shooting nodes and the control values over the shooting time intervals
 - At the beginning, we will give

$$\mathbf{x} = \underbrace{\{x_0, \dots, x_0\}}_{N+1 \text{ times}}, \quad \mathbf{u} = \underbrace{\{\emptyset_{n_u}, \dots, \emptyset_{n_u}\}}_{N \text{ times}}$$

(of course, this is an arbitrary choice and other options are available)

- For the next time instants, *acados* by default will take as initial guess the solution of the previous problem solved

While this is a reasonable choice, we can also think of another possibility, called *shifting*: since the new problem is solved $T_s = T/N$ seconds after the previous one, the best guess for the *first* state and the *first* input of the sequence is actually the one that was previously the *second* element of the sequence

⇒ the same reasoning applies for the following elements, so in the end we get the next initial guess by *shifting* the previous solution

⇒ this clearly works up to the second to last element of the sequence; then, for the last one, we can take it equal to the previous one



Note: We are implicitly exploiting the assumption that the shooting time intervals have all equal length T/N and the sample time of the controller T_s is set to T/N as well.

```
[ ]: # define simulation fundamental time step [s]
ts_sim = 0.001
```

```

# model used to simulate the system
sim_model = get_inverted_pendulum_model(l=0.8)
sim_model.name = "sim_model"

# initial condition
x0 = np.array([0, np.pi, 0, 0])

# setup controller parameters
# - system model
model = get_inverted_pendulum_model()
# - controller sample time [s]
Ts = 0.02
# - number of shooting time intervals
N = 100
# - prediction horizon length [s]
T = N*Ts

# get state and control dimensions
nx, nu = model.x.rows(), model.u.rows()

# define reference
# - define reference for the angle and, accordingly, the simulation time Tf
angle_ref, Tf = piecewise_constant(np.array([np.pi, 0]), np.array([5, 10]), Ts)

# - provide a reference for all variables
y_ref = np.hstack((np.zeros((len(angle_ref), 1)), angle_ref.reshape(-1,1), np.
    ↪ zeros((len(angle_ref), nx+nu-2))))

# - add N samples at the end (replicas of the last sample) for reference look-ahead
y_ref = np.vstack((y_ref, np.repeat(y_ref[-1].reshape(1,-1), N, axis=0)))

# compute the number of steps for simulation
N_steps, N_steps_dt, n_update = compute_num_steps(ts_sim, Ts, Tf)

# configure whether to apply shifting and to enable reference look-ahead
shifting = False
ref_preview = False

```

3.3.3 Closed-loop simulation

```

[ ]: clear_previous_simulation()

# setup simulation of system dynamics
sim = AcadosSim()
sim.model = sim_model
sim.solver_options.T = ts_sim
sim.solver_options.integrator_type = 'ERK'

```

```

acados_integrator = AcadosSimSolver(sim, verbose=False)

# create OCP solver
ocp = create_ocp_solver_description(model, N, T, x0)
acados_ocp_solver = AcadosOcpSolver(ocp, verbose=False)

# initialize solver
for stage in range(N):
    acados_ocp_solver.set(stage, "x", x0)
    acados_ocp_solver.set(stage, "u", np.zeros((nu,)))

acados_ocp_solver.set(N, "x", x0)

# define iteration counter for the discrete-time part of the control loop
k = 0

# create variables to store state and control trajectories
simX = np.zeros((N_steps + 1, nx))
simU = np.zeros((N_steps_dt, nu))
# set initial state
simX[0, :] = x0

# create variables to store, at each iteration, previous optimal solution
x_opt = np.zeros((N+1, nx, N_steps_dt))
u_opt = np.zeros((N, nu, N_steps_dt))

# variable to store total CPU time
cpt = np.zeros((N_steps_dt,))

# variable to store solver status
status = np.zeros((N_steps_dt,))

# simulation loop
for i in tqdm(range(N_steps), desc="Simulation", ascii=False, ncols=75, colour='green'):

    # check whether to update the discrete-time part of the loop
    if(i % n_update == 0):
        # update reference
        for j in range(N):
            acados_ocp_solver.set(j, "yref", y_ref[k + (j if ref_preview else 0), :])
            acados_ocp_solver.set(N, "yref", y_ref[k + (N if ref_preview else 0), 0:-nu])

        # if performing shifting, explicitly initialize solver
        # (otherwise, it will be automatically initialized with the previous solution)
        if shifting and k > 0:
            for stage in range(N):
                acados_ocp_solver.set(stage, "x", x_opt[stage+1, :, k-1])

```

```

        acados_ocp_solver.set(stage, "u", u_opt[min([stage+1,N-1]), :, k-1])

    acados_ocp_solver.set(N, "x", x_opt[N, :, k-1])

    # update the control
    simU[k, :] = acados_ocp_solver.solve_for_x0(simX[i, :],
                                                fail_on_nonzero_status=False,
                                                print_stats_on_failure=False)

    # store CPU time required for solving the problem
    cpt[k] = acados_ocp_solver.get_stats('time_tot')

    # store solver status
    status[k] = acados_ocp_solver.get_status()

    # store optimal solution
    for stage in range(N):
        x_opt[stage, :, k] = acados_ocp_solver.get(stage, "x")
        u_opt[stage, :, k] = acados_ocp_solver.get(stage, "u")

    x_opt[N, :, k] = acados_ocp_solver.get(N, "x")

    # update discrete-time iteration counter
    k += 1

    # simulate system
    simX[i + 1, :] = acados_integrator.simulate(simX[i,:], simU[k-1, :])

```

3.3.4 Visualize results

```

[ ]: print("Average total CPU time: " + str(np.mean(cpt)*1000) + " ms")

time      = np.linspace(0, ts_sim * N_steps,    N_steps    + 1)
time_dt   = np.linspace(0, Ts      * N_steps_dt, N_steps_dt + 1)

nonzero_status = np.argwhere(status != 0)

if nonzero_status.size != 0:
    print("\nSolver returned non-zero status at the following iterations:")
    for k in nonzero_status:
        print(f"* k = {int(k.item())} [t = {time_dt[k].item():.3f} s] - status_
↳ {int(status[k].item())}")

matplotlib.pyplot.close('all')
plot_results(time, time_dt, simX, simU)
plot_cpt(time_dt, cpt, Ts)

```

```
[ ]: plot_pred_traj(time, time_dt, simX, simU, x_opt, u_opt, np.
      ↳round(time_dt,3)==5.0))

[ ]: inverted_pendulum_animation(simX[:,0], simX[:, 1], ts_sim)

Video("simulation.mp4", width=640, height=480)
```

4 Use of slack variables

The constraints considered up to now are *hard*: they must be satisfied (within a prescribed tolerance, which is among the configuration parameters). However, in some cases it can be useful to be able to relax these constraints if needed, admitting a violation.

This can be done introducing **slack variables**, which are additional optimization variables for our problem. In particular, considering bounds like

$$l_{x_i} \leq x_i \leq u_{x_i}$$

which corresponds to

$$\begin{aligned} x_i &\leq u_{x_i} \\ -x_i &\leq -l_{x_i} \end{aligned}$$

they can be *softened* by introducing two slack variables ε_{l_i} and ε_{u_i} as follows:

- rewrite the constraints as

$$\begin{aligned} x_i - \varepsilon_{u_i} &\leq u_{x_i} \\ -x_i - \varepsilon_{l_i} &\leq -l_{x_i} \end{aligned}$$

- require the slack variables to assume a value greater than or equal to zero

$$\varepsilon_{u_i} \geq 0, \quad \varepsilon_{l_i} \geq 0$$

⇒ this ensures that, when the slack variables assume a value different from zero, they are indeed relaxing the constraints

- include in the cost function a the term accounting for the use of slack variables, so that the constraints are relaxed only “when necessary”

⇒ in particular, for each slack variable, we consider both a *quadratic* and a *linear* contribution to the cost

$$Z_{l_i} \varepsilon_{l_i}^2 + z_{l_i} \varepsilon_{l_i}, \quad Z_{u_i} \varepsilon_{u_i}^2 + z_{u_i} \varepsilon_{u_i}$$

Considering our example, we will now overwrite the function `create_ocp_solver_description`, that creates the code version of our problem formulation, introducing bounds on position (i.e., bounds on the first state component)

$$-1 \text{ m} \leq p \leq 1 \text{ m}$$

and we will make them *soft* with slack variables.

```
[ ]: def create_ocp_solver_description(model, N, T, x0) -> AcadosOcp:

    # create ocp object to formulate the OCP
    ocp = AcadosOcp()

    # define system dynamics model
```

```

ocp.model = model

# set prediction horizon:
# tf - prediction horizon length [s]
# N - number of intervals in which the prediction horizon is divided
ocp.solver_options.tf = T
ocp.solver_options.N_horizon = N

# NOTE: on older acados versions, use instead
# ocp.dims.N = N

# get state, control and cost dimensions
nx = model.x.rows()
nu = model.u.rows()

ny = nx + nu
ny_e = nx

# define cost type
ocp.cost.cost_type = 'LINEAR_LS'
ocp.cost.cost_type_e = 'LINEAR_LS'

# define cost weight matrices
Q = np.diag([10, 10, 0.1, 0.1])
R = 0.01

ocp.cost.W = block_diag(Q, R)
ocp.cost.W_e = T/N*Q

# define matrices characterizing the cost
ocp.cost.Vx = np.vstack((np.eye(nx), np.zeros((nu, nx))))
ocp.cost.Vu = np.vstack((np.zeros((nx, nu)), np.eye(nu)))
ocp.cost.Vx_e = np.eye(nx)

# alternatively, for the NONLINEAR_LS cost type
#ocp.model.cost_y_expr = ca.vertcat(model.x, model.u)
#ocp.model.cost_y_expr_e = model.x

# initialize variables for reference
ocp.cost.yref = np.zeros((ny,))
ocp.cost.yref_e = np.zeros((ny_e,))

# bounds on control input
ocp.constraints.idxbu = np.array([0])
ocp.constraints.lbu = np.array([-20])
ocp.constraints.ubu = np.array([20])

# bounds on position (0 component of state vector)

```

```

ocp.constraints.idxbx = np.array([0])
ocp.constraints.lbx = np.array([-1])
ocp.constraints.ubx = np.array([1])

# bounds on terminal state  $x_N$ 
ocp.constraints.idxbx_e = np.array([0])
ocp.constraints.lbx_e = np.array([-1])
ocp.constraints.ubx_e = np.array([1])

# indices among the bounds on state for which use a slack variable
ocp.constraints.idxsbx = np.array([0])
ocp.constraints.idxsbx_e = np.array([0])

# define weight on slack variables
ocp.cost.Zl = np.array([1e4])
ocp.cost.Zl_e = np.array([1e4])
ocp.cost.Zu = np.array([1e4])
ocp.cost.Zu_e = np.array([1e4])

ocp.cost.zl = np.array([1e3])
ocp.cost.zl_e = np.array([1e3])
ocp.cost.zu = np.array([1e3])
ocp.cost.zu_e = np.array([1e3])

# initialize constraint on initial condition
ocp.constraints.x0 = x0

# set solver options
ocp.solver_options.qp_solver = "PARTIAL_CONDENSING_HPIPM"
ocp.solver_options.hessian_approx = "GAUSS_NEWTON"
ocp.solver_options.integrator_type = "ERK"
ocp.solver_options.nlp_solver_type = "SQP"

# to configure partial condensing
#ocp.solver_options.qp_solver_cond_N = int(N/10)

# some more advanced settings (refer to the documentation to see them all)
# - maximum number of SQP iterations (default: 100)
ocp.solver_options.nlp_solver_max_iter = 100
# - maximum number of iterations for the QP solver (default: 50)
ocp.solver_options.qp_solver_iter_max = 50

return ocp

```

We can now use the previous cells to setup and run a simulation.

5 Use of non-uniform discretization grid

Up to now we have considered to define the shooting nodes in a *uniform* way: we have divided the prediction horizon T in N intervals of equal amplitude T/N , which we also assumed equal to the sample time T_s .

However, this not the only way of proceeding and, in particular, we can consider a **non-uniform grid**, using intervals of different lengths. $\Rightarrow$ in this way it is possible to cover a longer time horizon with a fewer number of shooting nodes, thus reducing the computational complexity

The grid can be defined freely, however to do it consider that:

- typically, an increasing interval length is chosen, with a grid that is therefore finer at the beginning of the prediction horizon
- at each discrete time step, we apply only the first control input of the optimal solution and then, after T_s seconds (the controller sample time) we solve a new problem
 - $\Rightarrow$ while up to now it was automatic that the control would be applied exactly for the time expected by the controller in the calculation, this is not guaranteed any more
 - $\Rightarrow$ what we do remains reasonably interpretable as long as we use intervals that have a minimum length of T_s

Note: The length of the shooting time intervals and the sample time of the controller can be different also using a uniform grid, i.e. with $T > T/N$, but recall that we assumed $T_s = T/N$.

With these considerations in mind, in the following we will use the following definition for our non-uniform grid over the prediction horizon $[0, T]$:

- we start from a *uniform* grid with T/T_s intervals
- we select a subset of m time instants from it, always including the first ($t = 0$) and the last ($t = T$) one
 - $\Rightarrow$ these will be the shooting nodes of our non-uniform grid, which is hence composed of $N = m - 1$ shooting time intervals

```
[ ]: def get_uniform_grid(T, N):  
  
    # compute length of each shooting interval  
    time_step = T/N  
  
    # compute the shooting nodes  
    shooting_nodes = np.linspace(0, time_step*N, N+1)  
  
    return shooting_nodes  
  
def get_nonuniform_grid(T, Ts, intermediate_nodes):  
  
    # start with a uniform grid  
    uniform_grid = get_uniform_grid(T, int(T/Ts))  
  
    # construct the list of indices to select, including the first and the last sample  
    indices = np.insert(intermediate_nodes, 0, 0)
```

```

indices = np.append(indices, -1)

# get the shooting nodes as the instants of the uniform grid
# corresponding to the selected indices
shooting_nodes = uniform_grid[indices]

return shooting_nodes

```

Let us consider a simple example, just creating and visualizing a uniform grid together with a non-uniform one:

```

[ ]: plot_grid(get_uniform_grid(2, 100), "Uniform grid")
plot_grid(get_nonuniform_grid(2, 0.02, [1, 2, 3, 5, 8, 13, 21, 34, 55]), "Non-uniform_
↪grid")

```

5.1 Modifying the problem definition

Now, in order to actually use the non-uniform grid in our Nonlinear MPC framework, we need to encode this information in the `AcadosOcp` object that we use and hence, we need once again to modify the function `create_ocp_solver_description`.

In particular, what we need to do is to set the field `solver_options.shooting_nodes` of the `AcadosOcp` object to an array containing the shooting nodes of our grid.

```

[ ]: def create_ocp_solver_description(model, N, T, x0, shooting_nodes) -> AcadosOcp:

    # create ocp object to formulate the OCP
    ocp = AcadosOcp()

    # define system dynamics model
    ocp.model = model

    # set prediction horizon:
    # tf - prediction horizon length [s]
    # N - number of intervals in which the prediction horizon is divided
    ocp.solver_options.tf = T
    ocp.solver_options.N_horizon = N

    # NOTE: on older acados versions, use instead
    # ocp.dims.N = N

    # set shooting nodes
    ocp.solver_options.shooting_nodes = shooting_nodes

    # get state, control and cost dimensions
    nx = model.x.rows()
    nu = model.u.rows()

    ny = nx + nu
    ny_e = nx

```

```

# define cost type
ocp.cost.cost_type = 'LINEAR_LS'
ocp.cost.cost_type_e = 'LINEAR_LS'

# define cost weight matrices
Q = np.diag([10, 10, 0.1, 0.1])
R = 0.01

ocp.cost.W = block_diag(Q, R)
ocp.cost.W_e = T/N*Q

# define matrices characterizing the cost
ocp.cost.Vx = np.vstack((np.eye(nx), np.zeros((nu, nx))))
ocp.cost.Vu = np.vstack((np.zeros((nx, nu)), np.eye(nu)))
ocp.cost.Vx_e = np.eye(nx)

# alternatively, for the NONLINEAR_LS cost type
#ocp.model.cost_y_expr = ca.vertcat(model.x, model.u)
#ocp.model.cost_y_expr_e = model.x

# initialize variables for reference
ocp.cost.yref = np.zeros((ny,))
ocp.cost.yref_e = np.zeros((ny_e,))

# set constraints
ocp.constraints.idxbu = np.array([0])
ocp.constraints.lbu = np.array([-20])
ocp.constraints.ubu = np.array([20])

# initialize constraint on initial condition
ocp.constraints.x0 = x0

# set solver options
ocp.solver_options.qp_solver = "PARTIAL_CONDENSING_HPIPM"
ocp.solver_options.hessian_approx = "GAUSS_NEWTON"
ocp.solver_options.integrator_type = "ERK"
ocp.solver_options.nlp_solver_type = "SQP"

# to configure partial condensing
#ocp.solver_options.qp_solver_cond_N = int(N/10)

# some more advanced settings (refer to the documentation to see them all)
# - maximum number of SQP iterations (default: 100)
ocp.solver_options.nlp_solver_max_iter = 100
# - maximum number of iterations for the QP solver (default: 50)
ocp.solver_options.qp_solver_iter_max = 50

```

```
return ocp
```

5.2 Setup simulation parameters

Finally, we can test this strategy in simulation. As before, we first need to define the parameters and in particular notice that now:

- we need to specify also the *shooting nodes* to create the non-uniform grid
- we are not considering *shifting* any more, as it was defined under the assumptions of uniform grid and controller sample time equal to the length of the shooting time intervals

```
[ ]: # define simulation fundamental time step [s]
ts_sim = 0.001

# model used to simulate the system
sim_model = get_inverted_pendulum_model(l=0.8)
sim_model.name = "sim_model"

# initial condition
x0 = np.array([0, np.pi, 0, 0])

# setup controller parameters
# - system model
model = get_inverted_pendulum_model()
# - controller sample time [s]
Ts = 0.02
# - prediction horizon length [s]
T = 2.0
# - indices of intermediate shooting nodes (between 0 and T) with respect to a uniform
    ↪ grid
intermediate_nodes = [1, 2, 3, 5, 8, 13, 21, 34, 55]
# - shooting nodes
shooting_nodes = get_nonuniform_grid(T, Ts, intermediate_nodes)
# - number of shooting time intervals
N = len(shooting_nodes) - 1

# get state and control dimensions
nx, nu = model.x.rows(), model.u.rows()

# define reference
# - define reference for the angle and, accordingly, the simulation time Tf
angle_ref, Tf = piecewise_constant(np.array([np.pi, 0]), np.array([5, 10]), Ts)

# - provide a reference for all variables
y_ref = np.hstack((np.zeros((len(angle_ref), 1)), angle_ref.reshape(-1,1), np.
    ↪ zeros((len(angle_ref), nx+nu-2))))

# - add T/Ts samples at the end (replicas of the last sample) for reference look-ahead
y_ref = np.vstack((y_ref, np.repeat(y_ref[-1].reshape(1,-1), int(T/Ts), axis=0)))
```

```

# compute the number of steps for simulation
N_steps, N_steps_dt, n_update = compute_num_steps(ts_sim, Ts, Tf)

# configure whether to enable reference look-ahead
ref_preview = False

```

5.3 Closed-loop simulation

And here we have the actual simulation. Notice in particular that also providing the *reference* to the controller at each discrete time step needs to properly take into account the fact that we are using a non-uniform grid.

```

[ ]: clear_previous_simulation()

# setup simulation of system dynamics
sim = AcadosSim()
sim.model = sim_model
sim.solver_options.T = ts_sim
sim.solver_options.integrator_type = 'ERK'

acados_integrator = AcadosSimSolver(sim, verbose=False)

# create OCP solver
ocp = create_ocp_solver_description(model, N, T, x0, shooting_nodes)
acados_ocp_solver = AcadosOcpSolver(ocp, verbose=False)

# initialize solver
for stage in range(N):
    acados_ocp_solver.set(stage, "x", x0)
    acados_ocp_solver.set(stage, "u", np.zeros((nu,)))

acados_ocp_solver.set(N, "x", x0)

# define iteration counter for the discrete-time part of the control loop
k = 0

# create variables to store state and control trajectories
simX = np.zeros((N_steps + 1, nx))
simU = np.zeros((N_steps_dt, nu))
# set initial state
simX[0, :] = x0

# create variables to store, at each iteration, previous optimal solution
x_opt = np.zeros((N+1, nx, N_steps_dt))
u_opt = np.zeros((N, nu, N_steps_dt))

```

```

# variable to store total CPU time
cpt = np.zeros((N_steps_dt,))

# variable to store solver status
status = np.zeros((N_steps_dt,))

# simulation loop
for i in tqdm(range(N_steps), desc="Simulation", ascii=False, ncols=75, colour='green'):

    # check whether to update the discrete-time part of the loop
    if(i % n_update == 0):
        # update reference
        if ref_preview:
            y_ref_k = y_ref[k:k+int(T/Ts)+1, :][[0] + intermediate_nodes + [-1]]
        else:
            y_ref_k = np.repeat(y_ref[k, :].reshape(1,-1), N+1, axis=0)

        for j in range(N):
            acados_ocp_solver.set(j, "yref", y_ref_k[j])
        acados_ocp_solver.set(N, "yref", y_ref_k[N, 0:-nu])

    # update the control
    simU[k, :] = acados_ocp_solver.solve_for_x0(simX[i, :],
                                                fail_on_nonzero_status=False,
                                                print_stats_on_failure=False)

    # store CPU time required for solving the problem
    cpt[k] = acados_ocp_solver.get_stats('time_tot')

    # store solver status
    status[k] = acados_ocp_solver.get_status()

    # store optimal solution
    for stage in range(N):
        x_opt[stage, :, k] = acados_ocp_solver.get(stage, "x")
        u_opt[stage, :, k] = acados_ocp_solver.get(stage, "u")

    x_opt[N, :, k] = acados_ocp_solver.get(N, "x")

    # update discrete-time iteration counter
    k += 1

# simulate system
simX[i + 1, :] = acados_integrator.simulate(simX[i, :], simU[k-1, :])

```

```
[ ]: print("Average total CPU time: " + str(np.mean(cpt)*1000) + " ms")

time      = np.linspace(0, ts_sim * N_steps,    N_steps      + 1)
time_dt    = np.linspace(0, Ts      * N_steps_dt, N_steps_dt  + 1)

nonzero_status = np.argwhere(status != 0)

if nonzero_status.size != 0:
    print("\nSolver returned non-zero status at the following iterations:")
    for k in nonzero_status:
        print(f"* k = {int(k.item())} [t = {time_dt[k].item():.3f} s] - status_␣
↪{int(status[k].item())}")

matplotlib.pyplot.close('all')
plot_results(time, time_dt, simX, simU)
plot_cpt(time_dt, cpt, Ts)

[ ]: plot_pred_traj(time, time_dt, simX, simU, x_opt, u_opt, np.argwhere(np.
↪round(time_dt,3)==5.0), shooting_nodes)
```